AI-Driven Wind Turbine Failure Prediction using Neural Networks

ReneWind and Deep Learning

April 19, 2026

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

Contents / Agenda

- Executive Summary
- Business Problem Overview and Solution Approach
- Data Overview
- Exploratory Data Analysis (EDA) Results
- Data Preprocessing
- Model Evaluation Criterion
- Model Performance Summary
- Test Performance of Best Model
- Data Background and Contents
- Appendix: (Univariate, Bivariate & Model Building) - Charts and Graphs

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

Executive Summary

Business Goal

- The objective is to build a neural network model to **predict wind turbine generator failures** using sensor data
- Multiple neural network architectures were built and compared across different configurations
- Models were progressively improved by adding **depth, dropout regularization, class weighting, and optimizer changes**

Key Insights

- The dataset is **heavily imbalanced** — only ~5.5% of cases are actual failures
- **Recall** was chosen as the evaluation metric to minimize missed failures
- **Adam optimizer** outperformed SGD in terms of recall and generalization

Final Outcome

- **Model 4** (Adam optimiser, 2 hidden layers, no dropout) was selected as the best-performing model

Business Recommendations

- Deploy Model 4 for real-time turbine monitoring to catch failures before they occur
- Prioritize early inspection when the model flags a potential failure to avoid costly replacements
- Improve calibration of sensors V1 and V2 which had missing values
- Periodically retrain the model as new sensor data accumulates

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

Business Problem Overview and Solution Approach

Problem Definition

- **The wind turbine monitoring process aims to:**
  - ✓ Predict generator failures before they occur
  - ✓ Reduce costly unplanned replacements
  - ✓ Minimise downtime through proactive maintenance
  - ✓ Support data-driven maintenance scheduling
- **This is a binary classification problem:**
  - ✓ **Target variable:** Failure Status (Failure = 1, No Failure = 0)
- **Cost structure:**
  - ✓ True Positive → Repair cost (medium)
  - ✓ False Negative → Full replacement (highest cost)
  - ✓ False Positive → Inspection only (lowest cost)

Solution Approach / Methodology

- Data Understanding & Overview
- Exploratory Data Analysis (EDA)
- Data Cleaning & Preprocessing
- Missing Value Imputation
- Train-Validation Split (stratified 80/20)
- Model Building — 7 Neural Network architectures
- Progressive improvement: depth → dropout → class weights → optimiser change
- Model Evaluation using Recall metric
- Final Model Selection based on validation performance

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

EDA Results – Univariate Analysis

Target Variable: Failure Status

- The dataset shows a strong class imbalance:
  - ✓ Majority class: ~94.5% No Failure (Class 0)
  - ✓ Minority class: ~5.5% Failure (Class 1)
- Similar distribution in test set — 94.36% vs 5.64%
- Confirms the need for Recall as the primary metric rather than Accuracy

Numerical Features (V1–V40)

- All 40 features are continuous and numerical (float64)
- Features are sensor readings — encrypted/ciphered for confidentiality
- Several features exhibit:
  - ✓ Skewed distributions
  - ✓ Outliers visible in boxplots
  - ✓ Wide variation in scale across features

Missing Values

- Missing values identified only in V1 and V2
- Present in both train and test datasets
- Treated using median imputation

Key Univariate Insights

- Strong class imbalance — accuracy alone would be misleading (~94% by predicting all zeros)
- Sensor features show varied distributions and outliers across all 40 variables
- Only 2 out of 40 features had missing values — data is largely complete

Link to – Appendix slide

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

EDA Results – Bivariate Analysis

Correlation Analysis

- Correlation heatmap computed across all 40 sensor features
- Some feature pairs show moderate correlation
- No extreme multicollinearity observed
- Neural networks handle correlated features well through learned weights

Class Distribution vs Features

- Failure cases (Class 1) represent a small but distinct minority
- The failure pattern is not random — it is learnable from sensor readings
- Features contribute independently to the prediction

Key Bivariate Insights

- No single feature perfectly separates failure from non-failure
- A combination of all 40 features is needed — justifying a neural network approach
- The low failure rate (~5.5%) makes Recall the critical success metric over Accuracy

Link to – Appendix slide

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

Data Preprocessing

Duplicate Value Check

- Checked for duplicate rows in the training dataset
- No duplicates found. All 20,000 rows are unique observations

Missing Value Detection & Treatment

- Missing values identified in V1 and V2 only
- Treated **using median imputation,** robust to outliers
- **Imputer** fitted only on X_train
- Applied separately to X_val and X_test to prevent data leakage
- Verified no missing values remain before modeling

Outlier Detection & Treatment

- Outliers detected via boxplots across all V1–V40 features
- No explicit removal applied
- Neural networks are relatively robust to outliers
- Removing outliers risks losing rare failure signals from the minority class

Feature Engineering

- No additional feature engineering performed
- All 40 sensor features used as is
- No encoding needed. All features are already numerical

Data Preparation for Modeling

Data Splitting:

- Training Set — 16,000 rows (80%)
- Validation Set — 4,000 rows (20%)
- Test Set — separate 5,000 rows held out for final evaluation
- Stratified split applied (random_state=1) to preserve class ratios in each split
- Class Imbalance Handling:
- Class weights computed from training labels
- Used in Models 3 and 6 to bias the model toward minority class (failures)

Final Insight

- Preprocessing ensured clean, properly structured data with no leakage, enabling fair and reliable model comparison

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

Model Evaluation Criterion

Common Settings Across All Models

- Epochs: 50
- Batch Size: 32
- Loss Function: **Binary Crossentropy**
- Output Layer: 1 neuron with **Sigmoid** activation
- Evaluation Metric: **Recall**

Model 0 — Baseline

- 1 Hidden Layer: 7 neurons, ReLU activation
- Optimizer: **SGD**
- Dropout: No
- Class Weight: No

Model 1 — Added Depth

- 2 Hidden Layers: 16 → 8 neurons, ReLU activation
- Optimizer: **SGD**
- Dropout: No
- Class Weight: No

Model 2 — Dropout Regularization

- 3 Hidden Layers: 16 → 8 → 4 neurons, ReLU activation
- Dropout: 50% after first hidden layer
- Optimizer: **SGD**
- Class Weight: No

Model 3 — Class Weighting

- 3 Hidden Layers: 16 → 8 → 4 neurons, ReLU activation
- Dropout: 50% after first hidden layer
- Optimizer: **SGD**
- Class Weight: Yes

Model 4 — Adam Optimiser (Best Model)

- 2 Hidden Layers: 16 → 8 neurons, ReLU activation
- Optimizer: **Adam**
- Dropout: No
- Class Weight: No

Link to – Appendix slide

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

Model Evaluation Criterion (cont.)

Model 5 — Deeper + Dropout + Adam

- 3 Hidden Layers: 32 → 16 → 8 neurons, ReLU activation
- Dropout: 50% after first hidden layer
- Optimizer: **Adam**
- Class Weight: No

Model 6 — Class Weight + Adam

- 3 Hidden Layers: 32 → 16 → 8 neurons, ReLU activation
- Dropout: 50% after first hidden layer
- Optimizer: **Adam**
- Class Weight: Yes
- Model 6 follows a similar architecture to Model 5 with the addition of class weighting
- The performance did not show a significant improvement over Model 4
- Adding class weights on top of an already well-performing Adam optimizer did not yield meaningful gains in this case
- Model 4 remained the stronger choice based on validation recall

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

Model Performance Summary

Key Observations

- Models progressively improved as depth, regularization and optimizer changes were introduced
- Model 0 established a solid baseline with a single hidden layer
- Model 1 showed a clear recall improvement simply by adding a second hidden layer
- Model 2 introduced dropout which helped regularization but slightly reduced recall
- Model 3 recovered recall by incorporating class weights to handle the imbalance
- Model 4 achieved the highest validation recall by switching to the Adam optimizer
- Model 5 added more depth and dropout with Adam but did not surpass Model 4
- Model 6 introduced class weighting on top of Model 5 however the performance remained comparable with no significant improvement over Model 4

Training Set Performance

Model	Accuracy	Recall	Precision	F1 Score
Model 0	0.9865	0.8916	0.9764	0.9294
Model 1	0.9921	0.9354	0.9879	0.9600
Model 2	0.9868	0.8849	0.9876	0.9294
Model 3	0.9666	0.9341	0.8183	0.8656
Model 4	**0.9942**	**0.9497**	**0.9946**	**0.9710**
Model 5	0.9913	0.9239	0.9929	0.9554
Model 6	0.9913	0.9239	0.9929	0.9554

Validation Set Performance

Model	Accuracy	Recall	Precision	F1 Score
Model 0	0.9868	0.8976	0.9725	0.9314
Model 1	0.9885	0.9176	0.9698	0.9419
Model 2	0.9873	0.8936	0.9827	0.9331
Model 3	0.9633	0.9191	0.8064	0.8523
Model 4	**0.9913**	**0.9318**	**0.9835**	**0.9559**
Model 5	0.9893	0.9095	0.9866	0.9443
Model 6	0.9893	0.9095	0.9866	0.9443

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

Best Model - Test Performance

Key Observations

- Model 0 — Simple baseline; decent recall but limited by shallow architecture
- Model 1 — Adding a second layer noticeably improved recall
- Model 2 — Dropout reduced overfitting but slightly lowered recall
- Model 3 — Class weights helped recover recall on imbalanced data
- Model 4 — Switching to Adam optimizer gave the highest validation recall
- Model 5 — Deeper with dropout; slightly lower recall than Model 4
- Model 6 — Excluded due to notebook bug

Final Model Selected: Model 4

- Architecture: Input (40) → Hidden Layer 1 (16, ReLU) → Hidden Layer 2 (8, ReLU) → Output (1, Sigmoid)
- Optimizer: Adam
- **Loss**: Binary Crossentropy
- Epochs: 50 | Batch Size: 32

Reason for Selection

- Highest validation recall of 93.18% — catches the most actual failures on unseen data
- Minimal overfitting — train-validation recall gap of only 1.8%
- Best balance between recall, precision, and generalization
- Adam optimizer converges faster and more effectively than SGD for this problem

Final Insight

- **Model 4** generalizes well — test recall (93.17%) closely matches validation recall (93.18%), confirming no overfitting
- The model successfully identifies 93 out of every 100 actual turbine failures
- Directly reduces costly unplanned generator replacements
- Suitable for real-world deployment as a predictive maintenance decision-support tool

Final Model Performance on Test Set

Metric	Value
Accuracy	99.13%
Recall	93.17%
Precision	98.35%
F1 Score	95.59%

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

Data Background and Contents

Data Background

- The dataset represents sensor readings collected from wind turbines operated by ReneWind
- Objective: To predict whether a wind turbine generator will fail based on sensor readings, so repairs can be done before a breakdown occurs
- This is a classification problem with:
  - ✓ Target variable: Target
  - ✓ Classes: Failure (1), No Failure (0)

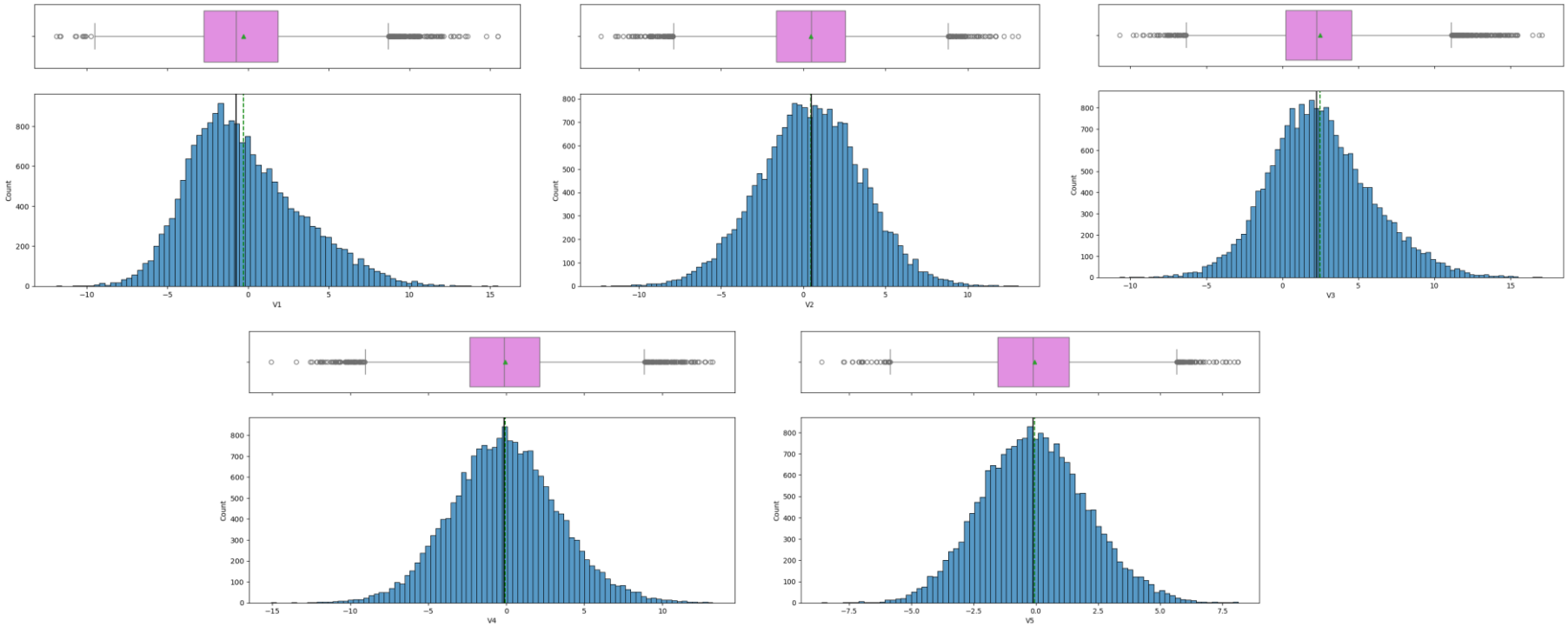
Dataset Overview

- Contains only numerical features
- Data reflects:
  - ✓ Sensor readings from various turbine components
  - ✓ Environmental and mechanical measurements
- All feature values are continuous (float64)
- Features are ciphered — exact sensor type is confidential

Feature	Description
V1 – V10	Ciphered sensor reading - environmental or mechanical measurement from the turbine
V11 – V40	Ciphered sensor readings - same nature as above, covering additional turbine parameters
Target	Binary flag indicating generator status - 1 = Failure, 0 = No Failure

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

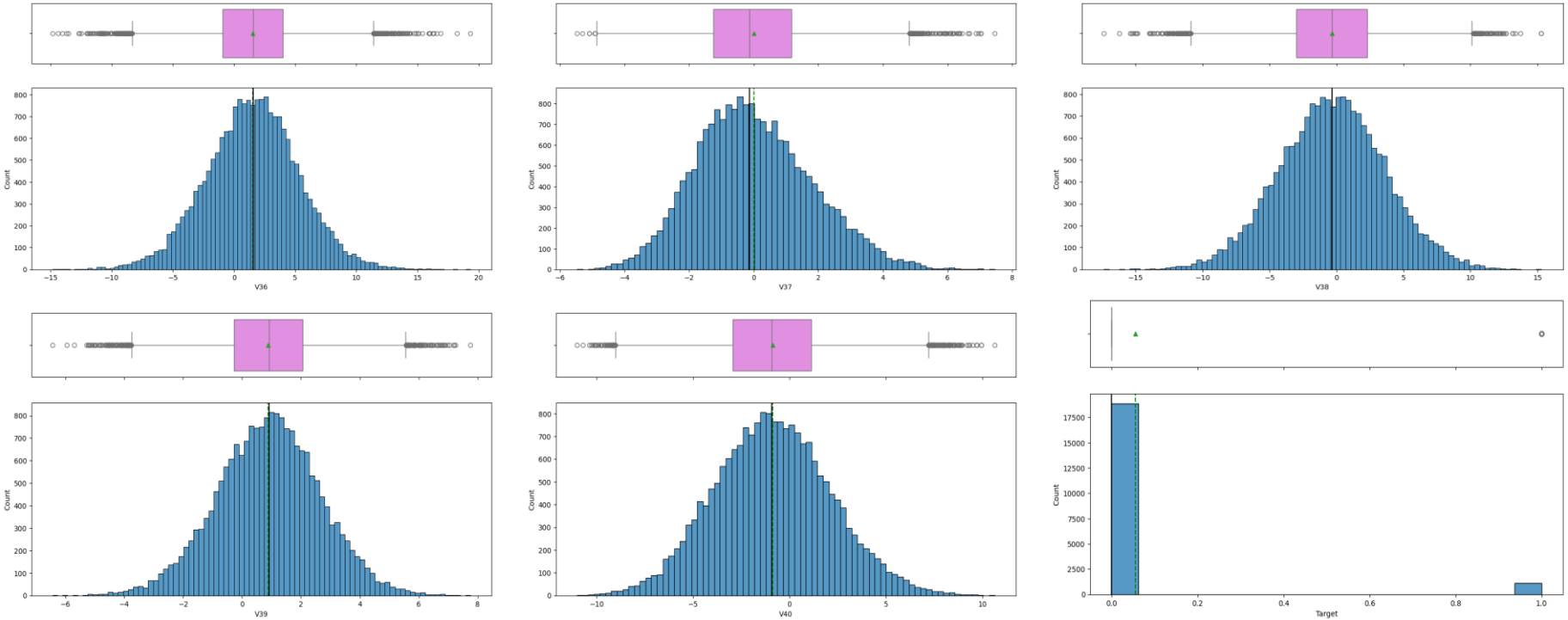
Univariate Analysis – Histogram & Box Plot (V1-V5)



APPENDIX

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

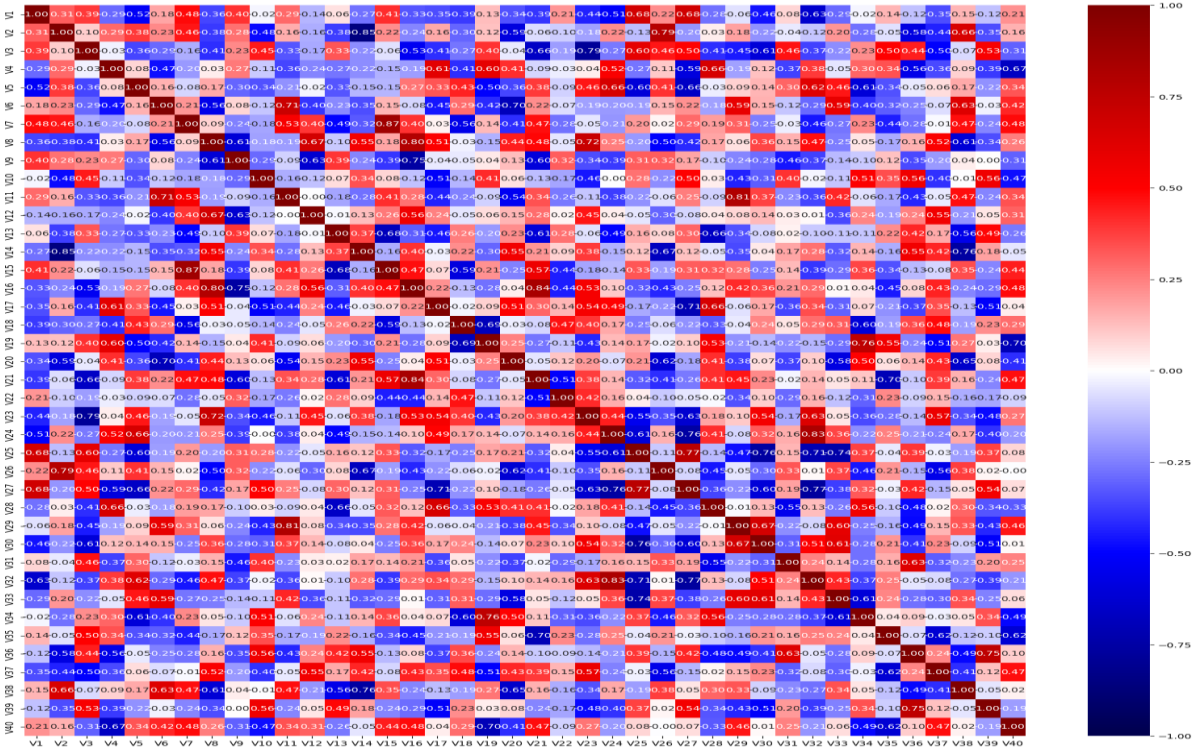
Univariate Analysis - Histogram & Box Plot (V36-V40 + Target)



APPENDIX

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

Bivariate Analysis – Correlation Heatmap

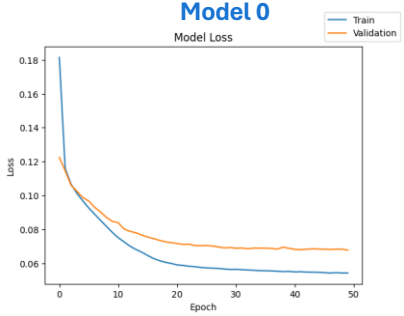


APPENDIX

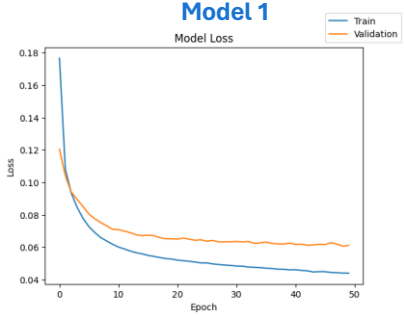
Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

Model Building - Graphs

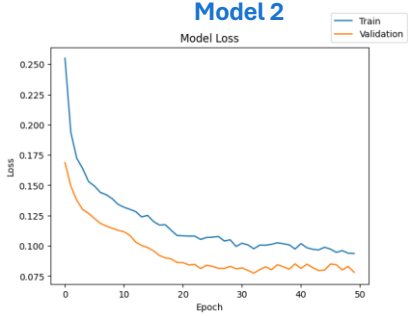
Model 0



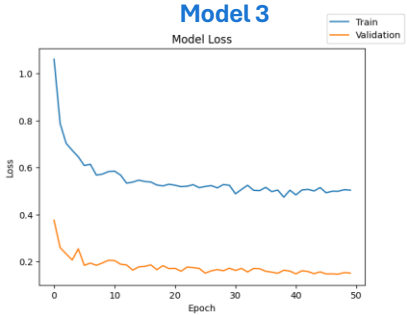
Model 1



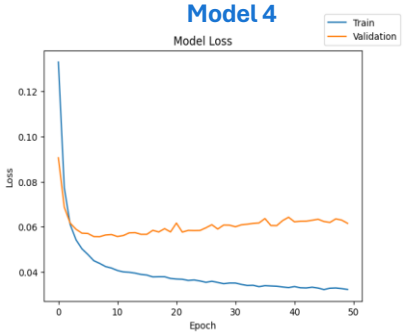
Model 2



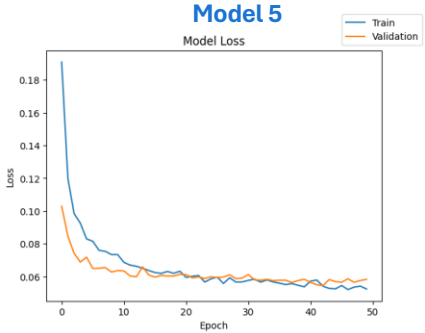
Model 3



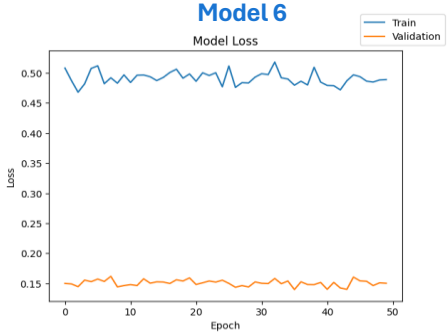
Model 4



Model 5



Model 6



APPENDIX

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

Univariate Charts

Additional univariate analysis visualizations were performed but are not included due to space constraints as they exceed over 30 charts

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

Great Learning

Happy Learning !